

JJ VPS System Guide

Item	Value
Document Title	JJ VPS System Guide
Project	JJ_VPS
Document Type	System Guide
Version	Draft 1.0
Last Updated	2026-05-16
Primary Scope	Runtime architecture, provider integration, localization flow, audio interaction, and extension guidance

Table of Contents

- 1. [Purpose](#)
 - 2. [System Overview](#)
 - 3. [Architecture Overview](#)
 - 4. [Major Modules](#)
 - 1. [Jorjin SDK Layer](#)
 - 2. [Unified VPS Layer](#)
 - 3. [Immersal SDK Path](#)
 - 4. [MultiSet SDK Path](#)
 - 5. [Localized Map Context Layer](#)
 - 6. [RAG Voice Interaction Layer](#)
 - 7. [Agent Guidance Layer](#)
 - 5. [End-to-End Runtime Flows](#)
 - 6. [Scene Setup Guidance](#)
 - 7. [Configuration and Inspector Notes](#)
 - 8. [Extension Strategy](#)
 - 9. [Appendix](#)
 - 10. [Conclusion](#)
-

1. Purpose

This document describes the current system design of JJ_VPS, with emphasis on how the project combines:

- Jorjin device SDK integration
- provider-agnostic VPS orchestration
- Immersal and MultiSet localization routes
- Firebase-backed localized map context
- RAG-based voice interaction
- Agent-based proactive and reactive guidance

It is intended for:

- developers maintaining the runtime pipeline
- technical leads reviewing integration boundaries
- implementers setting up scenes and prefabs
- handoff documentation for future extension

2. System Overview

2.1 Product Summary

JJ_VPS is a Unity application for AR glasses or camera-assisted spatial interaction. The system captures image data from Jorjin hardware or debug sources, runs VPS localization through either Immersal or MultiSet, aligns spatial content in Unity, then layers voice-based interaction and AI guidance on top of the localized scene.

2.2 Core Capabilities

The current codebase supports:

1. Jorjin display and camera access through JJ SDK wrappers.
2. A unified IVpsProvider architecture that can route to Immersal or MultiSet.
3. Single-shot or repeated localization through VpsLocalizationController.
4. MultiSet scene alignment through `SingleFrameLocalizationManager.mapSpace`.
5. Firebase-based loading of per-map POI metadata after localization.
6. World-space label rendering for localized POIs.
7. Voice recording through MicController.
8. RAG question-answer flow through RAGController.
9. Agent-based proactive guidance and user-initiated scene-aware questions.

2.3 Key Design Intent

The project already has several useful abstraction boundaries:

- IVpsImageSource
- IVpsProvider
- IVpsWorldAligner

- IAgentTextService
- IAgentSpeechService

These boundaries are the main reason the project can support both Immersal and MultiSet under one controller while also allowing RAG and Agent interaction layers to coexist.

3. Architecture Overview

3.1 Layered View

The system can be understood in six layers:

1. **Device Layer**

JJ SDK display, camera, sensor, and hardware-facing wrappers.

2. **Image Acquisition Layer**

Converts camera or debug image sources into normalized VPS requests.

3. **Localization Routing Layer**

Routes requests to either Immersal or MultiSet.

4. **World Alignment and Spatial Content Layer**

Applies or respects provider-owned world alignment and loads localized scene content.

5. **Context and Metadata Layer**

Resolves localized mapName, loads Firebase POI data, and spawns map labels.

6. **Voice and AI Interaction Layer**

Handles voice recording, RAG flow, waiting/response audio, and Agent guidance.

3.2 High-Level Component Map

JJCameraManager / CamRenderer
-> CamRendererVpsImageSource
-> VpsLocalizationController
 -> ImmersalVpsProviderAdapter
 -> MultiSetVpsProviderAdapter
 -> IVpsWorldAligner / provider-owned alignment

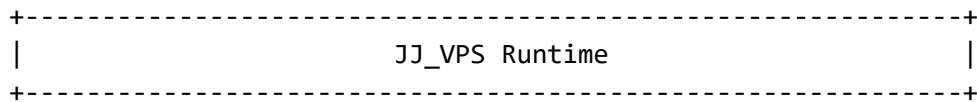
MultiSet SingleFrameLocalizationManager
-> mapSpace alignment
-> map mesh loading
-> LocalizationSnapshot

LocalizedMapContextController
-> MultiSetApiManager.GetMapDetails
-> Firebase /Marks.json
-> LocalizedMapLabels

MicController
-> RAGController
-> AgentController

AgentController
-> LocalizedMapContextController
-> SingleFrameLocalizationManager
-> IAgentTextService
-> IAgentSpeechService

3.3 Architecture Diagram



[Jorjin SDK Layer]

JJCameraManager -----> CamRenderer -----> RawImage preview
 JJDisplayManager -----> JJ glasses display mode / brightness

|
v

[Image Acquisition Layer]

CamRendererVpsImageSource
 - reads CamRenderer.camBytes or debug image
 - attaches intrinsics
 - attaches optional query pose
 - outputs VpsLocalizationRequest

|
v

[Unified VPS Routing Layer]

VpsLocalizationController
 - localizeOnStart
 - autoLocalize
 - provider selection
 - success/failure events

|
|
v

|
|
v

[Immersal Path]

ImmersalVpsProviderAdapter
 -> ImmersalAPI

[MultiSet Path]

MultiSetVpsProviderAdapter
 -> SingleFrameLocalizatio
 -> mapSpace alignment
 -> mesh loading
 -> LocalizationSnapshot

\
\
\
v

/
/
/
v

[Localized Map Context Layer]

LocalizedMapContextController
 -> resolve mapName
 -> query Firebase /Marks.json
 -> build CurrentContext
 -> spawn LocalizedMapLabels

|

·
v

[Voice / AI Interaction Layer]

MicController

- > RAGController (RAG path)
- > AgentController (Agent path)

AgentController

- > scene-aware prompt
- > passive guidance
- > active user question
- > TTS playback

3.4 Runtime Responsibility Split

- CamRenderer or other camera sources produce raw frame data.
 - CamRendererVpsImageSource normalizes image bytes and intrinsics.
 - VpsLocalizationController decides when to localize and which provider to use.
 - MultiSetVpsProviderAdapter or ImmersalVpsProviderAdapter forwards the request.
 - SingleFrameLocalizationManager or ImmersalAPI returns provider-specific localization results.
 - LocalizedMapContextController resolves the current map and POI metadata.
 - MicController, RAGController, and AgentController build the voice interaction layer on top.
-

4. Major Modules

4.1 Jorjin SDK Layer

Representative source files:

- Assets/Scripts/JJSDK/Camera/JJCameraManager.cs
- Assets/Scripts/JJSDK/Display/JJDisplayManager.cs
- Assets/Scripts/CamRenderer.cs

4.1.1 Responsibility

This layer abstracts Jorjin Android-side services into Unity-friendly wrappers.

4.1.2 JJCameraManager

JJCameraManager is a thin wrapper around `com.jorjin.jjsdk.camera.CameraManager`. It handles:

- opening and stopping the glasses camera
- choosing resolution
- registering frame listeners
- exposing camera parameters

This class is a device-facing service, not a scene component.

4.1.3 JJDisplayManager

JJDisplayManager wraps `com.jorjin.jjsdk.display.DisplayManager` and controls:

- display open/close
- 2D or 3D display mode
- brightness
- auto-brightness behavior

It does not choose which Unity texture is shown. The actual rendered content still comes from the Unity camera/XR output path.

4.1.4 CamRenderer

CamRenderer is the project's runtime bridge between camera frames and Unity scene usage.

It supports:

- live JJ camera frames
- a debug image path via `debugMode`
- UI preview through `RawImage` `image`
- `camBytes` output for downstream VPS flow

This makes it usable both for real device testing and offline localization tests.

4.2 Unified VPS Layer

Representative source files:

- `Assets/Scripts/VPS/IVpsProvider.cs`
- `Assets/Scripts/VPS/VpsLocalizationController.cs`
- `Assets/Scripts/VPS/CamRendererVpsImageSource.cs`
- `Assets/Scripts/VPS/WorldAlignment/TransformVpsWorldAligner.cs`
- `Assets/Scripts/VPS/WorldAlignment/NoOpVpsWorldAligner.cs`

4.2.1 Responsibility

This layer provides a provider-agnostic contract for localization.

4.2.2 CamRendererVpsImageSource

This component converts current camera state into a `VpsLocalizationRequest`.

It is responsible for:

- taking image bytes from `CamRenderer`
- optionally reading `CamRenderer.image.texture` in debug mode
- JPEG encoding
- attaching intrinsics
- attaching query camera pose when available

4.2.3 VpsLocalizationController

This is the main runtime orchestration point for localization. It handles:

- provider selection via `VpsProviderType`
- `localizeOnStart`
- `autoLocalize`
- repeated trigger timing
- success and failure events

Important behavior:

- `autoLocalize` can now be disabled at runtime and stops immediately for future cycles.
- if a provider reports `providerAppliedWorldPoseInternally = true`, controller-level world alignment is skipped.

4.2.4 World Aligner Variants

`TransformVpsWorldAligner` is the generic world-pose applier for providers that return a pose but do not directly mutate scene transforms.

`NoOpVpsWorldAligner` is appropriate when the provider already handles world alignment internally.

In current `MultiSet` usage, the common pattern is to use provider-owned alignment and therefore either:

- rely on `NoOpVpsWorldAligner`, or
 - leave a world aligner assigned but unused because the provider reports internal application.
-

4.3 Immersal SDK Path

Representative source files:

- `Assets/Scripts/ImmersalCustom/ImmersalAPI.cs`
- `Assets/Scripts/VPS/Providers/ImmersalVpsProviderAdapter.cs`

4.3.1 Responsibility

This path wraps the existing Immersal workflow inside the unified `IVpsProvider` architecture.

4.3.2 ImmersalAPI

`ImmersalAPI` is the project's legacy/localized implementation for Immersal. It handles:

- encoded image localization requests
- base64 payload generation
- parsing Immersal responses
- position and rotation application for the player object path
- `LocalizationCompleted` event emission

It also writes user pose values into `RAGController.pos_x`, `pos_z`, and `rot`, which is part of the current RAG interaction path.

4.3.3 ImmersalVpsProviderAdapter

This adapter converts the generic `VpsLocalizationRequest` into the payload expected by `ImmersalAPI`.

It also converts `ImmersalLocalizationSnapshot` into the normalized `VpsLocalizationResult` structure.

4.3.4 Architectural Note

Immersal remains compatible with the new VPS architecture, but its alignment semantics differ from `MultiSet`. Immersal tends to be more tightly coupled with the existing player/XR transform route, while `MultiSet` currently drives `mapSpace`.

4.4 MultiSet SDK Path

Representative source files:

- `Assets/MultiSet/Script/SingleFrameLocalizationManager.cs`
- `Assets/Scripts/VPS/Providers/MultiSetVpsProviderAdapter.cs`

- `Assets/MultiSet/Script/MapMeshHandler.cs`
- `Assets/MultiSet/Script/ShaderProgressAnimator.cs`

4.4.1 Responsibility

This path uses MultiSet as the active VPS backend and aligns a Unity mapSpace root object to the localized result.

4.4.2 SingleFrameLocalizationManager

This is the main runtime entry point for MultiSet single-frame localization.

It supports:

- AR-session-based localization
- externally supplied image bytes and intrinsics
- repeated localization attempts
- confidence filtering
- background localization
- map or mapset targeting via `mapOrMapsetCode`
- success/failure snapshots and debug metrics

Important runtime outputs:

- `LatestLocalization`
- `LocalizationCompleted`
- `mapSpace`
- success/failure timing
- `mapName`
- localized camera pose
- applied map-space pose

4.4.3 Scene Alignment Strategy

In the current architecture, MultiSet success typically moves:

- `mapSpace`

instead of moving:

- the player
- the AR camera

This means the environment is aligned around the user rather than the user being repositioned in the world.

4.4.4 MultiSetVpsProviderAdapter

This adapter:

- maps generic intrinsics into `CameraParams`
- maps image size into `MultiSet.Resolution`
- forwards the image to
`SingleFrameLocalizationManager.LocalizeImageBytes(...)`
- converts `LocalizationSnapshot` into `VpsLocalizationResult`

It explicitly reports:

```
providerAppliedWorldPoseInternally = true
```

which is why controller-level world alignment is skipped for this route.

4.4.5 Mesh and Visualization

On successful localization, `MultiSet` can also:

- fetch map details
- load associated mesh content
- apply a transparent or animated shader treatment
- add `ShaderProgressAnimator`

This means `MultiSet` is not only a pose provider, but also part of the localized content-loading flow.

4.5 Localized Map Context Layer

Representative source files:

- `Assets/Scripts/MapContext/LocalizedMapContextController.cs`
- `Assets/Scripts/MapContext/WorldSpaceBillboard.cs`
- `Assets/Scripts/MapContext/FirebaseConfig.cs`

4.5.1 Responsibility

This layer converts a successful localization into scene-semantic context.

4.5.2 LocalizedMapContextController

This component listens to

`SingleFrameLocalizationManager.LocalizationCompleted`. When localization succeeds, it:

1. resolves `mapName`
2. queries `Firebase /Marks.json`

3. finds entries associated with that map
4. loads POI records under the matching title(s)
5. builds CurrentContext
6. spawns world-space labels under mapSpace

4.5.3 Data Imported Per POI

Each LocalizedMapPoiRecord currently stores:

- label
- localPosition
- localScale
- margin
- angle1
- angle2
- keyword
- details
- sourceTitle

4.5.4 Runtime Use

This POI data is shared by:

- label rendering
- Agent passive trigger logic
- Agent active question prompt context
- future AI scene reasoning

4.5.5 Rendering Notes

Labels are currently implemented as world-space UI and can be configured to render on top using a custom overlay material path.

4.6 RAG Voice Interaction Layer

Representative source files:

- Assets/Scripts/RAGAPI/MicController.cs
- Assets/Scripts/RAGAPI/RAGController.cs
- Assets/Scripts/RAGAPI/WavUtility.cs

4.6.1 Responsibility

This layer handles microphone recording and the direct RAG question-answer path.

4.6.2 MicController

MicController is the common microphone front-end for both RAG and Agent voice workflows.

Current behavior:

- records microphone input
- saves `input.wav`
- supports device selection
- routes requests by `VoiceRequestTarget`

Possible targets:

- RAG
- Agent

4.6.3 Event-Driven Audio Feedback

MicController is intentionally event-driven for audio cues:

- `onRequestStarted`
- `onRequestSent`

This allows scene-level configuration of:

- start sound
- waiting sound
- request-sent sound

without hard-coding audio behavior into the script.

4.6.4 RAGController

RAGController owns the existing RAG service workflow. It handles:

- STT selection
- TTS selection
- upload to external gateway
- playing response audio
- optional OpenAI STT/TTS paths
- waiting sound through `StartProcessingSound()` and `StopProcessingSound()`

The RAG flow is the correct route when the interaction should go to the external RAG backend rather than the local Agent scene reasoning path.

4.7 Agent Guidance Layer

Representative source files:

- `Assets/Scripts/Agent/AgentController.cs`
- `Assets/Scripts/Agent/AgentApiSettings.cs`
- `Assets/Scripts/Agent/IAgentTextService.cs`
- `Assets/Scripts/Agent/IAgentSpeechService.cs`
- `Assets/Scripts/Agent/OpenAITextService.cs`
- `Assets/Scripts/Agent/OpenAITtsSpeechService.cs`
- `Assets/Scripts/Agent/LocalSpeechService.cs`
- `Assets/Scripts/Agent/FallbackSpeechService.cs`

4.7.1 Responsibility

This layer provides two interaction modes:

- passive guidance triggered by spatial context
- active user-initiated questions based on current scene state

4.7.2 AgentController

`AgentController` is the orchestration point for:

- reading current localized map context
- computing user-to-POI distance and direction
- determining near vs inside state
- writing user state into TMP UI fields
- constructing prompts
- generating text
- playing speech
- emitting `OnResponseReady`

4.7.3 Passive Guidance

Passive guidance observes current player state and triggers when:

- a POI is spatially relevant
- the player remains stable for `stableSeconds`
- cooldown has elapsed
- the same POI has not just been handled

The trigger can use:

- per-POI `Firestore` margin
- per-POI `Firestore` `angle1/angle2`

with global Inspector values as fallback.

4.7.4 Active Questioning

When MicController routes to Agent, the voice path is:

1. record audio
2. OpenAI transcription
3. `AgentController.AskQuestion(...)`
4. prompt generation using current scene context
5. TTS playback

The active question prompt includes:

- current player status
- localized map name and id
- user position in world and map-local space
- user facing yaw
- all current POI relations

This path is intentionally scene-aware rather than target-only.

4.7.5 UI Outputs

AgentController currently supports optional TMP outputs for:

- `simpleMSGText`
- `nearestOutputText`
- `directionOutputText`
- `insideOutputText`
- `mapNameText`
- `agentResponseText`
- `playerMapPoseText`

This makes the Agent layer useful not only for speech, but also for debugging and operator-facing overlays.

4.7.6 Audio Flow

For Agent route, the recommended sound design is:

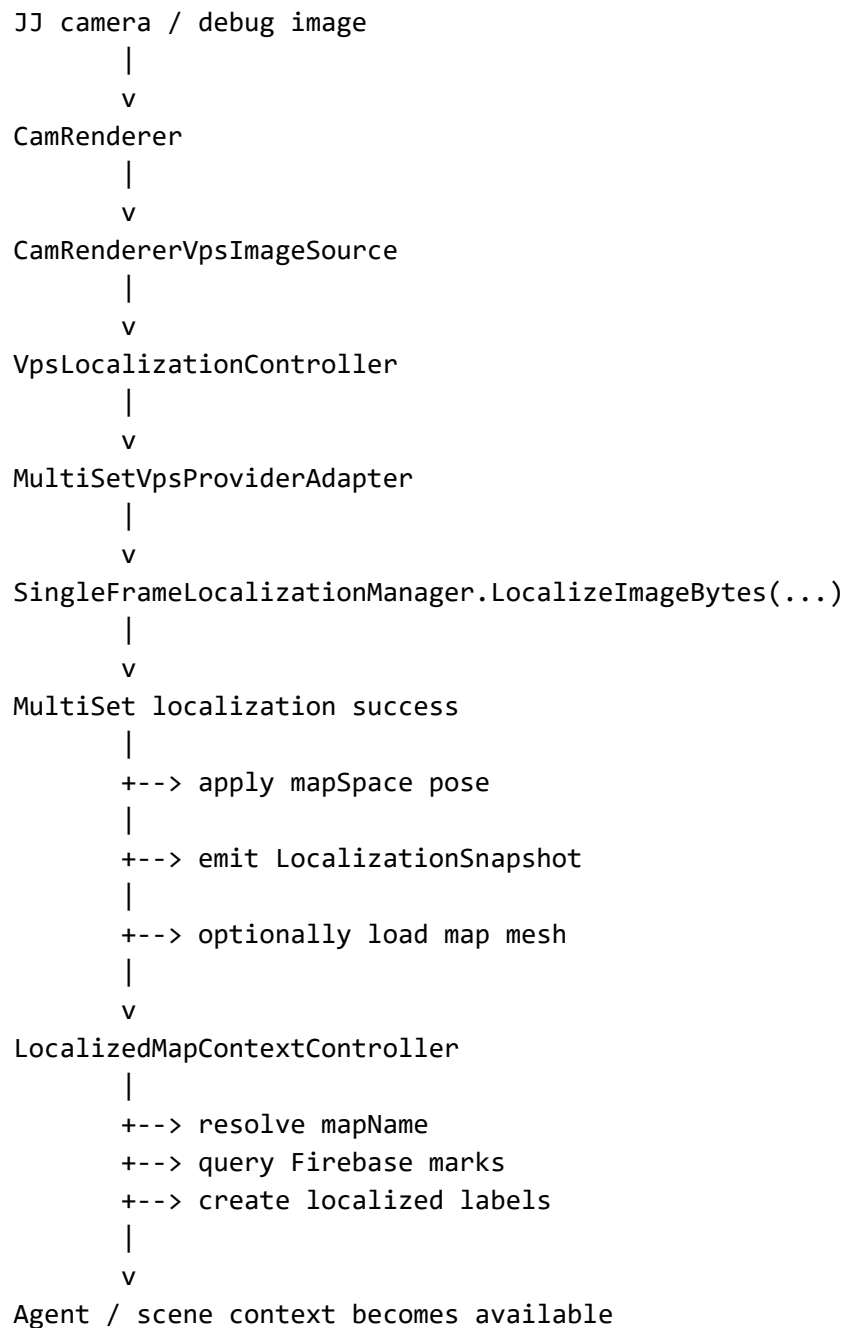
- `MicController.onRequestStarted` -> start cue
- `MicController.onRequestSent` ->
`AgentController.StartProcessingSound`
- `AgentController.OnResponseReady` -> short response-ready cue
- `AgentController.audioSource` -> TTS answer playback

4.7.7 Architectural Note

The current Agent implementation no longer depends on the old `EditorManager` or `MarkStorage`. It has already been refactored to use the new localization and Firebase map context path.

5. End-to-End Runtime Flows

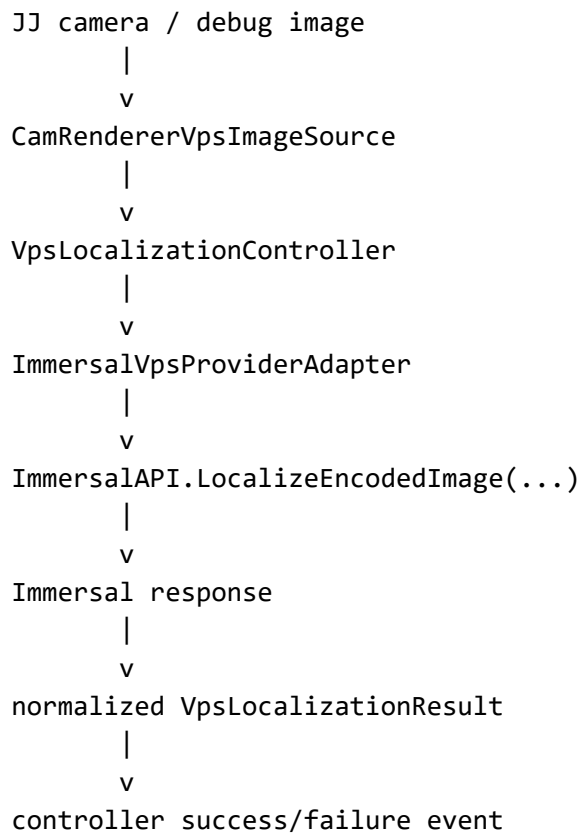
5.1 MultiSet Localization Flow



1. `CamRenderer` or debug image source produces frame data.

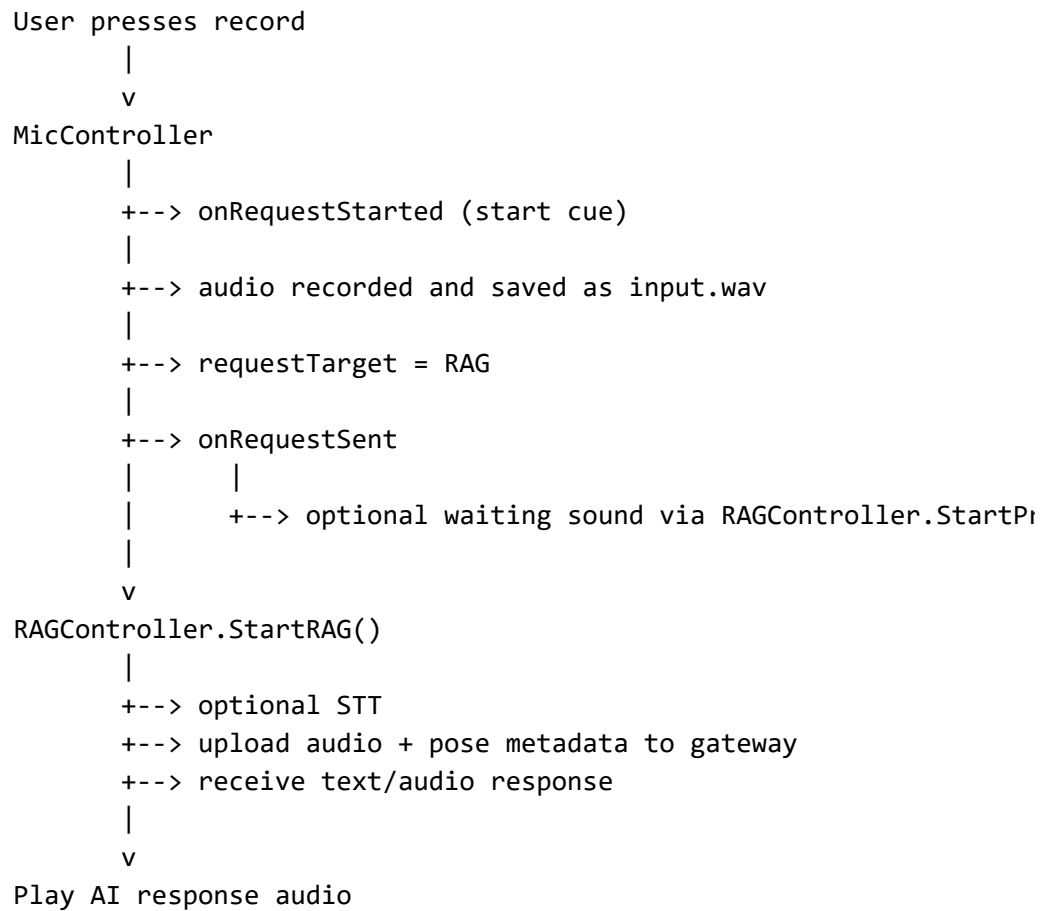
2. `CamRendererVpsImageSource` builds a `VpsLocalizationRequest`.
3. `VpsLocalizationController` routes to `MultiSetVpsProviderAdapter`.
4. `MultiSetVpsProviderAdapter` calls
`SingleFrameLocalizationManager.LocalizeImageBytes(...)`.
5. `MultiSet` localizes against `mapOrMapsetCode`.
6. `SingleFrameLocalizationManager` applies `mapSpace`.
7. Localization snapshot is published.
8. `LocalizedMapContextController` resolves map metadata and Firebase POIs.
9. Labels and Agent scene context become available.

5.2 Immersal Localization Flow



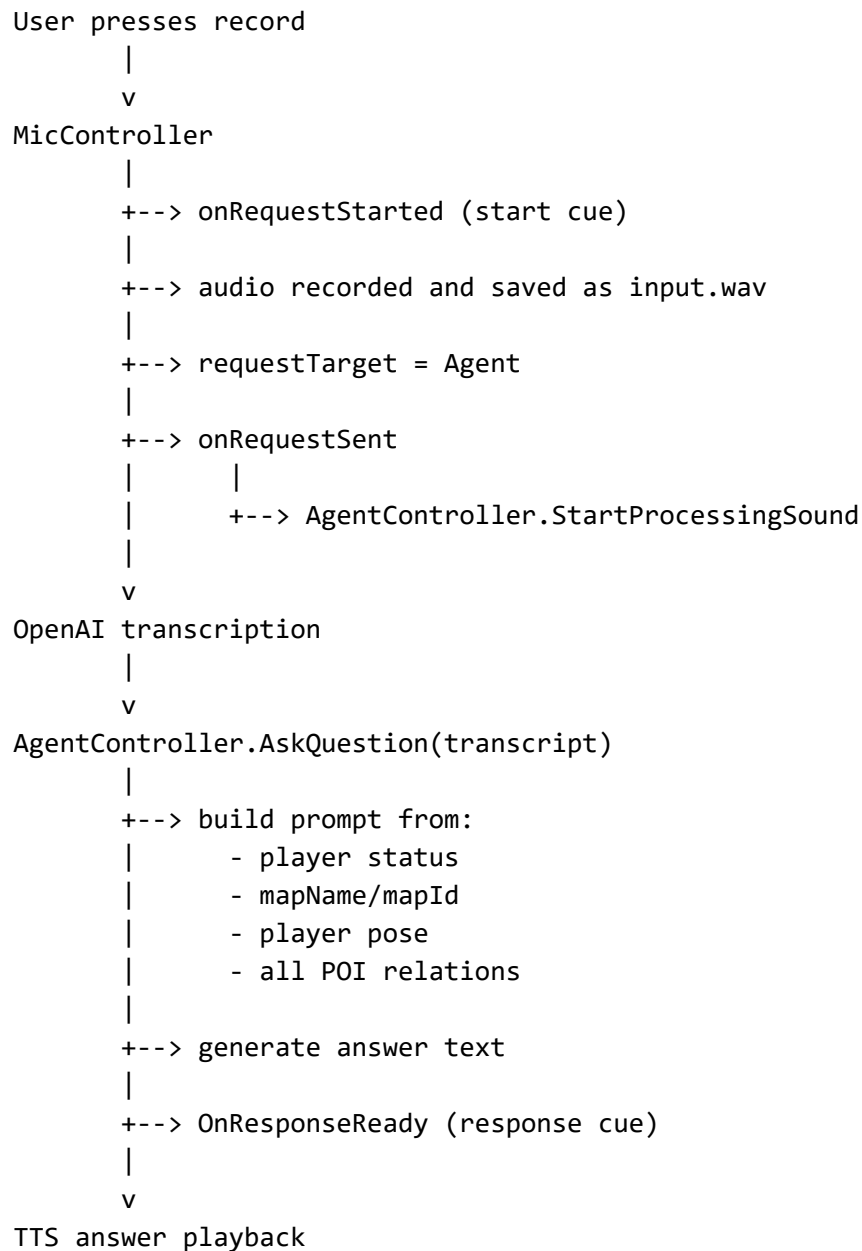
1. Generic image request is built.
2. `VpsLocalizationController` routes to `ImmersalVpsProviderAdapter`.
3. Adapter sends encoded image to `ImmersalAPI`.
4. Immersal returns localization data.
5. Result is emitted into the normalized provider event path.

5.3 RAG Voice Flow



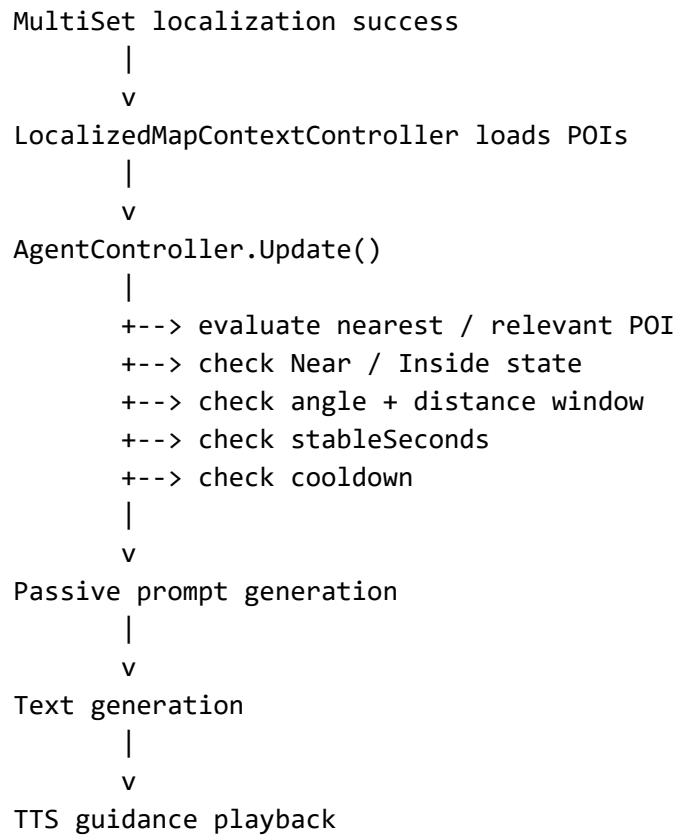
1. User records through MicController.
2. requestTarget = RAG.
3. RAGController.StartRAG() uploads audio and metadata.
4. RAG service returns text/audio.
5. RAGController plays answer audio.

5.4 Agent Voice Flow



1. User records through MicController.
2. requestTarget = Agent.
3. Audio is transcribed through OpenAI STT.
4. AgentController.AskQuestion(...) receives the transcript.
5. Prompt is constructed using current scene context.
6. Agent generates answer text.
7. OnResponseReady fires.
8. TTS speech is played.

5.5 Passive Guidance Flow



1. Localization succeeds and labels are available.
 2. AgentController evaluates player state every frame.
 3. A stable near/inside condition is detected.
 4. Passive prompt is generated.
 5. Guidance text is generated and spoken.
-

6. Scene Setup Guidance

6.1 Minimum MultiSet Scene Setup

Recommended scene components:

- SingleFrameLocalizationManager
- MultiSetVpsProviderAdapter
- CamRenderer
- CamRendererVpsImageSource
- VpsLocalizationController
- LocalizedMapContextController
- AgentController
- MicController

6.2 Core References

Typical reference wiring:

- `CamRendererVpsImageSource.camRenderer` -> current `CamRenderer`
- `CamRendererVpsImageSource.poseCamera` -> AR camera or player camera
- `MultiSetVpsProviderAdapter.localizationManager` -> `SingleFrameLocalizationManager`
- `VpsLocalizationController.imageSourceBehaviour` -> `CamRendererVpsImageSource`
- `VpsLocalizationController.multiSetProviderBehaviour` -> `MultiSetVpsProviderAdapter`
- `LocalizedMapContextController.localizationManager` -> `SingleFrameLocalizationManager`
- `AgentController.mapContextController` -> `LocalizedMapContextController`
- `AgentController.localizationManager` -> `SingleFrameLocalizationManager`
- `AgentController.userCamera` -> current user-facing camera
- `MicController.agentController` -> `AgentController`
- `MicController.ragController` -> `RAGController`

6.3 Audio Setup Recommendation

Use separate audio sources for:

- microphone start cue
- waiting loop
- response-ready cue
- TTS playback

This avoids conflicts between looped waiting audio and spoken answers.

7. Configuration and Inspector Notes

7.1 MultiSet Configuration

Important `SingleFrameLocalizationManager` fields:

- `mapOrMapsetCode`
- `localizationType`
- `useARSessionForLocalization`
- `autoLocalize`
- `firstLocalizationUntilSuccess`

7.2 VPS Automation

Important VpsLocalizationController fields:

- `localizeOnStart`
- `autoLocalize`
- `autoLocalizeInterval`

Runtime behavior:

- disabling `autoLocalize` during play now stops future automatic cycles immediately

7.3 Agent Configuration

Important AgentController fields:

- `apiSettings.apiKey`
- `processingAudioSource`
- `audioSource`
- passive trigger values
- TMP debug fields

7.4 Mic Configuration

Important MicController fields:

- `requestTarget`
- `recordButton`
- `microphoneDropdown`
- `agentController`
- `ragController`

For Agent STT, MicController now reads:

- `AgentController.apiSettings.apiKey`

directly, rather than maintaining a separate Inspector key field.

7.5 Firebase Configuration

Firebase entry point is:

- `Assets/Scripts/MapContext/FirebaseConfig.cs`

with the database URL centralized in `FirebaseConfig.DatabaseUrl`.

8. Extension Strategy

8.1 Adding Another VPS Provider

Recommended approach:

1. implement `IVpsProvider`
2. keep request shape aligned with `VpsLocalizationRequest`
3. emit normalized `VpsLocalizationResult`
4. plug the provider into `VpsLocalizationController`

This allows scene automation and Agent logic to remain unchanged.

8.2 Replacing or Extending Image Sources

Implement another `IVpsImageSource` when localization should consume:

- a device camera
- a `RenderTarget`
- screenshots
- offline test assets

8.3 Extending Agent Providers

To integrate another LLM or TTS provider:

1. implement `IAgentTextService`
2. implement `IAgentSpeechService`
3. wire the implementation into `AgentController.InitializeServices()`

8.4 Stabilization Recommendation

The current codebase would benefit from additional shutdown-safety around async localization follow-up flows, especially:

- Firebase async map context loading
- map-detail callbacks
- mesh loading during play-mode teardown

This is the main area to prioritize if editor stability during stop/recompile becomes a recurring issue.

9. Appendix

9.1 Key Source Files

- `Assets/Scripts/JJSDK/Camera/JJCameraManager.cs`
- `Assets/Scripts/JJSDK/Display/JJDisplayManager.cs`

- Assets/Scripts/CamRenderer.cs
- Assets/Scripts/VPS/VpsLocalizationController.cs
- Assets/Scripts/VPS/CamRendererVpsImageSource.cs
- Assets/Scripts/VPS/Providers/MultiSetVpsProviderAdapter.cs
- Assets/Scripts/VPS/Providers/ImmersalVpsProviderAdapter.cs
- Assets/MultiSet/Script/SingleFrameLocalizationManager.cs
- Assets/Scripts/ImmersalCustom/ImmersalAPI.cs
- Assets/Scripts/MapContext/LocalizedMapContextController.cs
- Assets/Scripts/RAGAPI/MicController.cs
- Assets/Scripts/RAGAPI/RAGController.cs
- Assets/Scripts/Agent/AgentController.cs
- Assets/Scripts/Agent/AgentApiSettings.cs

10. Conclusion

JJ_VPS is no longer just a direct VPS demo. It is a layered runtime that combines device integration, provider abstraction, spatial localization, localized metadata loading, and AI voice interaction in one project.

The most important architectural idea in the current codebase is the separation between:

- image acquisition
- localization provider routing
- world alignment ownership
- localized map context
- voice interaction and AI guidance

That separation is what makes the system maintainable and what allows Immersal, MultiSet, RAG, and Agent functionality to coexist in one scene-driven application.